

ПОСТУПАШКИ • КАРЬЕРА

СОБЕСЕДОВАНИЕ • АЛГОРИТМЫ

Задачи по алгоритмам в Яндексе

Реальные задачи с алгоритмических секций — с разбором подхода, типичными ошибками и примерами решений.



t.me/postypashki_old

t.me/postypashki_old

t.me/postypashki_old

39

задач с
разбором

8

тем: строки, массивы, деревья и др.

37

тегов для навигации по типам задач

C++

примеры решений из исходника

ПОСТУПАШКИ • КАРЬЕРА

Как пользоваться файлом

В этом файле собраны алгоритмические задачи с собеседований. К каждой задаче — описание, разбор подхода, типичные ошибки, а где-то и пример решения.

Задачи разбиты по темам, и **каждая тема начинается с новой страницы**. У каждой карточки есть теги — по ним удобно ориентироваться, какой тип задачи перед вами. Содержание кликабельно.

Содержание

Строки	15 задач	стр. 3
Два указателя	3 задачи	стр. 13
Массивы	6 задач	стр. 16
Хэш-таблицы и множества	4 задачи	стр. 21
Сортировка и интервалы	4 задачи	стр. 25
Поиск и слияние	2 задачи	стр. 28
Деревья	2 задачи	стр. 31
Проектирование и структуры данных	3 задачи	стр. 33

Здесь нет грейдов и таймингов — это чистые алгоритмы. Важнее идея решения, его чистота и понимание, где обычно ошибаются.

#Строки

Строки

Парсинг, преобразования и обработка строк — самый большой блок алгоритмических секций.

15 задач · описание, разбор подхода и типичные ошибки

1 Разворачивание скобочной грамматики в строку

#строки

#стек

#рекурсия

#парсинг

Дана скобочная грамматика: в круглых скобках записано выражение, следом за ними идут квадратные скобки с неотрицательным целым числом. Разворачивается в виде конкатенации строки в круглых скобках на саму себя N раз, где N - число в квадратных скобках.

Чуть более формально, грамматика:

```
term ::= a-z*
```

```
term ::= (term)[\d+]
```

`term ::= term1term2` - разворачивается в конкатенацию двух выражений. Если видно что кандидат не справляется с задачей, можно убрать это правило вывода без него задача решается значительно легче в виде простой линейной рекурсии.

На вход подается строка со скобочным выражением. Необходимо развернуть его в строку и вернуть. Предлагается считать, что скобочная последовательность на входе всегда корректна (не хочется здесь решать задачу валидации).

Примеры:

С двумя правилами:

```
" " -> " "
```

```
"ab" -> "ab"
```

```
"(ab)[3]" -> "ababab"
```

```
"((ab)[2])[2]" -> "abababab"
```

```
"(())[1][2]" -> ""
```

С тремя правилами:

```
"(a)[2](b)[2]" -> "aabb"
```

```
"((a)[2]b)[3]" -> "aabaabaab"
```

```
"()[1]bc" -> "bc"
```

```
"(a)[0]bc" -> "bc"
```

```
"abc(d)[2]" -> "abcdd"
```

Что требуется

- Аккуратное решение со стеком либо рекурсией.
- Можно сперва разделить выражение на конкатенацию термов, потом вычислить значение каждого терма. Вычисление терма `()[]` будет рекурсивно звать основную функцию для вычисления своего подвыражения в круглых скобках.

Где ошибаются

- Подводные камни:
- выход за границы строки при разборе, неточное вычленение содержимого подвыражений в скобках.
- остаются куски грамматики в выводе (например, когда есть третье правило недоразобран второй терм в `abc(a)[2]`).
- некорректно парсится число в квадратных скобках, если имеет более одного разряда — например, парсится задом наперёд или только один разряд.

Похожие задачи для тренировки

- LeetCode 394 *Decode String* (синтаксис `k[str]`, почти та же) — <https://leetcode.com/problems/decode-string/>

2 URL-кодирование

#строки #два-указателя #inplace

Написать метод, который заменит все пробелы в строке на `"%20"` inplace.

На вход подается строка с зарезервированными под расширение символами (гарантируется, что `resize()` до разумных размеров не будет выделять память).

Ограничения: $O(1)$ по памяти, $O(N)$ по времени, менять исходную строку можно.

Что требуется

- Два прохода по строке:
- На первом считаем кол-во пробелов, и таким образом итоговую длину.
- На втором идём с конца и копируем символы, кодируя в процессе пробелы.

Где ошибаются

- Пытаются копировать слева направо и затирают ещё не обработанные символы.

Похожие задачи для тренировки

- <https://aaronpkelly.github.io/2021/06/04/crackingTheCodingInterview/interviewQuestions1.3.html>
- Идеино: LeetCode 151 *Reverse Words in a String* (inplace с конца) — <https://leetcode.com/problems/reverse-words-in-a-string/>

3 Длинное умножение на разряд

#строки #арифметика #inplace

Дана строка из десятичных цифр (длинное число, младшие разряды расположены по младшему индексу). Написать код, который умножит это число на число $1 \leq n \leq 9$.

Разновидность той же задачи: в качестве "цифр" используются 4-байтовые беззнаковые числа (`ui32`), а умножение делаем на число, уместяющееся в `ui32`.

Что требуется

- Ограничения по памяти: $O(1)$ дополнительной памяти, т.е. надо использовать исходную строку (считаем, что возможное увеличение длины на 1 разряд не приведёт к реаллокации).

Где ошибаются

- Перенос (carry) на последнем разряде, забытый финальный перенос.

Похожие задачи для тренировки

- LeetCode 43 *Multiply Strings* (умножение двух длинных чисел) — <https://leetcode.com/problems/multiply-strings/>
- LeetCode 66 *Plus One* (тот же класс с переносом) — <https://leetcode.com/problems/plus-one/>

4 Изменения порядка слов в строке без изменения положения пробелов

#строки

#два-указателя

Написать функцию, которая меняет порядок слов в строке на противоположный, при этом не меняя расположение пробелов. Например: `hello_my__dear_world__ -> _world_dear____my_hello_` (пояснение: `_` это пробел)

Что требуется

- Хочется увидеть максимально простой работающий код. Хорошо, если он учитывает нюансы вроде пустой строки на входе, строки только из пробелов, строки вообще без пробелов.
- Можно поговорить про происходящие аллокации.

Где ошибаются

- Смещают пробелы; падают на краевых случаях (пустая строка / только пробелы / без пробелов).

Похожие задачи для тренировки

- LeetCode 151 *Reverse Words in a String* — <https://leetcode.com/problems/reverse-words-in-a-string/>
- LeetCode 186 *Reverse Words in a String II* — <https://leetcode.com/problems/reverse-words-in-a-string-ii/>
- Сохранение позиций пробелов — авторское усложнение.

5 Восстановить IP-адрес(a)

t.me/postypashki_old

#строки

#бэктрекинг

t.me/postypashki_old

t.me/postypashki_old

Дана строка из цифр.

Восстановить все возможные варианты IPv4-адресов, которые при удалении точек дали бы эту строку.

Что требуется

- Пойдёт самое простое решение с бэктрекингом, хорошо, если кандидат расскажет каких вроде нормальных IP-адресов быть не может.

Где ошибаются

- Ведущие нули (`01`), октеты > 255 , неверная длина строки.

Похожие задачи для тренировки

- LeetCode 93 *Restore IP Addresses* (один в один) — <https://leetcode.com/problems/restore-ip-addresses/>

6 Количество подстрок без повторов символов

#строки

#скользящее-окно

#два-указателя

#хэш-таблица

Дана строка символов. Найти количество пар индексов i и j ($i < j$), между которыми, включительно, нет повторяющихся символов.

Пример: Для строки "aba" ответ 5:

`[0, 0] ("a")``[0, 1] ("ab")`

```
[1, 1] ("b")
[1, 2] ("ba")
[2, 2] ("a")
```

Что требуется

- Решение за $O(N)$ по времени, $O(|\text{Alphabet}|)$ по памяти, где N - длина строки, $|\text{Alphabet}|$ - размер алфавита.
- Делаем один проход по строке с запоминанием последней виденной позиции символа в хэш-таблице.

Где ошибаются

- Последняя виденная позиция используется как есть, тогда как строка без повторений может начинаться только со следующего символа.
- Используется последняя виденная позиция для текущего символа, а не самая правая позиция последнего повторяющегося символа. В итоге в строке типа "baab" подстрока "aab" будет ошибочно считаться не имеющей повторов.

Похожие задачи для тренировки

- LeetCode 3 *Longest Substring Without Repeating Characters* (тот же приём) — <https://leetcode.com/problems/longest-substring-without-repeating-characters/>

7 Поиск подстроки в строке без учета порядка букв

#строки #скользящее-окно #хэш-таблица

Дан текст T и строка S . Требуется найти подстроку S' в T такую, что она совпадает с S с точностью до перестановки букв.

В качестве ответа стоит вернуть индекс первого вхождения, или -1 , если такая подстрока S' не нашлась.

Что требуется

- Два варианта решения: за $O(|T| \cdot |\Sigma|)$ и за $O(|T|)$, где Σ — алфавит. Второе чуть сложнее первого, до него некоторые не догадываются.
- Для AA-секции обязательным требованием является решение за $O(|T|)$, где константа не должна зависеть от размера алфавита.
- Первое решение: представляем S в качестве набора частот букв, потом идём по строке и сравниваем частоты в окне с референсными. Код для этого варианта лучше не писать.
- Второе решение: представление строки в виде хэш-таблицы с диффом частот между S и текущим окном. Тут нужен код.

Где ошибаются

- Пересчитывают всё окно каждый шаг (лишний множитель $|\Sigma|$), не удерживают дифф частот.

Похожие задачи для тренировки

- LeetCode 438 *Find All Anagrams in a String* — <https://leetcode.com/problems/find-all-anagrams-in-a-string/>
- LeetCode 567 *Permutation in String* — <https://leetcode.com/problems/permutation-in-string/>

8 Кратчайшее расстояние между элементами массива

#строки

#один-проход

Условие

Дана строка, состоящая из букв 'X', 'Y' и 'O'. Необходимо найти кратчайшее расстояние между буквами 'X' и 'Y', либо вывести 0, если 'X' либо 'Y' отсутствуют.

Примечания

- дистанция между двумя рядом стоящими буквами считается как 1 (одно межбуквенное расстояние)
- дистанция может считаться в обе стороны (взаимное расположение ближайших друг к другу X и Y не имеет значения)

Примеры

```
"YY" -> 0
"XX" -> 0
"XY" -> 1
"YXX" -> 2
"OXXOXYOX" -> 2
"OXXOXY" -> 2
```

Что требуется

- Один проход по массиву с запоминанием последних встреченных позиций X и Y и текущего кратчайшего расстояния.

Где ошибаются

- Учитывают только одно направление; не обрабатывают отсутствие одной из букв.

Похожие задачи для тренировки

- LeetCode 243 *Shortest Word Distance* (то же со словами) — <https://leetcode.com/problems/shortest-word-distance/>
- LeetCode 245 *Shortest Word Distance III* — <https://leetcode.com/problems/shortest-word-distance-iii/>

9 Удалить смайлики вида ":-)))" и ":-((" из сообщения

#строки

#автомат

#один-проход

Есть сообщения из соцсети, например:

```
"Я работаю в Гугле :-)))"
"везет :-) а я туда собеседование завалил:-(("
"лол:)"
"Ааааа!!!! :-))(()"
```

Хочется удалить из них смайлики, подпадающие под регулярку `"[:-\)+|:-\(+"` за линейное время. То есть, сделать так:

```
"Я работаю в Гугле
"везет а я туда собеседование завалил"
"лол:)"
"Ааааа!!!! (())"
```

Специально обрабатывать вложенные смайлики нельзя. Так, `: - : - ) ) ( ( (` должно превратиться в `: - ( ( (`, но не в пустую строку.

Что требуется

- Можно быстро обсудить регулярки / sed, но требуем написать алгоритм руками.
- Нужен однопроходный алгоритм а-ля поддержание состояния автомата, например, текущей длины заматчившегося смайлика.
- Есть альтернативное решение на базе идеи «пытаться приложить смайл на каждую позицию в строке».
- Опционально можно попросить константу доп. памяти.

Где ошибаются

- Некоторые варианты реализации автомата довольно массивны по коду, из-за чего можно не успеть всё написать. Отчасти это возникает из-за «что и когда писать в выход». В таких случаях не возбраняется помочь кандидату упростить автомат.

Похожие задачи для тренировки

- LeetCode 65 *Valid Number* (конечный автомат по строке) — <https://leetcode.com/problems/valid-number/>
- Прямого клона нет — авторская.

10 Проверка строк на исправление (OneEditApart)

#строки

#два-указателя

Нужно реализовать функцию `OneEditApart`, проверяющую, можно ли одну строку получить из другой не более, чем за одно исправление (удаление, добавление, изменение символа):

```
OneEditApart("cat", "dog") -> false
OneEditApart("cat", "cats") -> true
OneEditApart("cat", "cut") -> true
OneEditApart("cat", "cast") -> true
OneEditApart("cat", "at") -> true
OneEditApart("cat", "acts") -> false
```

Что требуется

- Решение за $O(n)$ и с $O(1)$ доп. памяти.

Где ошибаются

- Дублируют логику для веток «длиннее / короче»; неверная обработка разной длины строк.

Похожие задачи для тренировки

- LeetCode 161 *One Edit Distance* — <https://leetcode.com/problems/one-edit-distance/>

11 Сумма хекс-чисел

#строки

#арифметика

Даны две строки, каждая из которых — хекс-число в прямой записи (старший знак в нулевой позиции). Можно считать, что запись корректна (строки не пустые, не содержат ведущих нулей и состоят только из символов от 0 до 9 и от a до f).

Написать код суммирования этих чисел.

Пример: `sum("fed", "13") == "1000"`

Что требуется

- Сложение столбиком.

```
std::string sum(const std::string& l, const std::string& r);  
  
assert(sum("fed", "13") == "1000");
```

Где ошибаются

Стандартные ошибки:

1. FF + FF
2. FFFFF + 1

Похожие задачи для тренировки

- LeetCode 415 *Add Strings* (то же для десятичных) — <https://leetcode.com/problems/add-strings/>
- LeetCode 67 *Add Binary* (бинарный вариант) — <https://leetcode.com/problems/add-binary/>

12 Нормализация пробелов

#строки

#два-указателя

#inplace

Заменить последовательности пробелов в строке на одиночные пробелы. $O(n)$ по времени, $O(1)$ по памяти.

```
normalize("some   string ")  
"some string "
```

Считается, что строка в языке mutable, если это не так, то можно считать, что в функцию передается изменяемый массив с символами.

Что требуется

- Простой in-place алгоритм с двумя указателями.

Где ошибаются

- забывает изменить размер строки
- неправильно обрабатываются пробелы в конце строки
- обработка пустой строки
- пробелы в начале строки

Похожие задачи для тренировки

- Идеино: LeetCode 151 *Reverse Words in a String* (там тоже сжимают пробелы) — <https://leetcode.com/problems/reverse-words-in-a-string/>
- Точного аналога нет — авторская.

13 Реализовать функцию fuzzysearch

#строки

#подпоследовательность

#два-указателя

Реализовать функцию `fuzzysearch` как в редакторе `sublime text 3`.

Для незнакомых с редактором и/или термином `fuzzysearch` (нечёткий поиск), можно упомянуть более формальное название `approximate string matching` (нахождение приблизительного соответствия строк). По факту требуется проверить, является ли первая строка подпоследовательностью второй.

Пример работы:

```
1 fuzzysearch('car', 'cartwheel');  
// true  
2 fuzzysearch('cwhl', 'cartwheel');  
// true  
3 fuzzysearch('cwheel', 'cartwheel');  
// true  
4 fuzzysearch('cartwheel', 'cartwheel');  
// true  
5 fuzzysearch('cweheel', 'cartwheel');  
// false  
6 fuzzysearch('lw', 'cartwheel');  
// false
```

Что требуется

- Хочется, чтобы кандидат задавал вопросы про тесткейсы: 1) пустая строка?, 2) регистр?, 3) 'wl' (подстрока, не начинающаяся на ту же букву, что строка).
- Хотим увидеть реализацию функции с использованием только одного прохода по символам строки, без использования `regex`.

Где ошибаются

- Тянут `regex` вместо простого однопроходного сканирования; не уточняют краевые тест-кейсы.

Похожие задачи для тренировки

- LeetCode 392 *Is Subsequence* (ровно она) — <https://leetcode.com/problems/is-subsequence/>

14 Сжатие строки

#строки

#один-проход

Дана строка, состоящая из букв A-Z:

"AAAABBBCCXYZDDDDDEEEFFFAAAAAABBBBBBBBBBBBBBBBBBBBBBBBBB"

Нужно написать функцию RLE, которая на выходе даст строку вида:

"4A382C1X1Y1Z4D3E3F6A28B"

И сгенерирует любую ошибку, если на вход пришла невалидная строка.

Пояснения:

К каждому символу добавляется количество его повторений; (можно сделать чтобы если символ один то не добавлять кол-во повторений)

Сперва указываем количество повторений, затем сам символ.

Что требуется

- Один проход по строке; корректно выводить последнюю группу и генерировать ошибку на невалидном входе.

- Тест-кейсы: пустая строка; 1 символ или 2 разных символа; несколько правильных символов; недопустимая строка.

Где ошибаются

- забыл проверить наличие одного символа
- забыл вывод последней группы
- забыл проверить наличие 1 в последней группе
- забыл сгенерировать ошибку

Пример решения

```
public static String countString(String s) {
    if (s.isEmpty() || s == null) {
        throw new IllegalArgumentException("input string is not valid");
    }

    StringBuilder result = new StringBuilder();
    int count = 1;

    for (int i = 0; i < s.length(); i++) {
        if (i + 1 != s.length() && s.charAt(i) == s.charAt(i + 1)) {
            count++;
        } else {
            result.append(count).append(s.charAt(i));
            count = 1;
        }
    }

    return result.toString();
}
```

t.me/postypashki_old

t.me/postypashki_old

t.me/postypashki_old

Похожие задачи для тренировки

- LeetCode 443 *String Compression* (in-place, близко) — <https://leetcode.com/problems/string-compression/>
- CodeRun 310 *Кодирование длин серий* (RLE, прямое соответствие) — <https://coderun.yandex.ru/problem/rle-test>

15 Исключить символ из строки и получить палиндром

#строки

#два-указателя

На входе дана непустая строка. Требуется выяснить, можно ли удалить из нее ровно один символ так, чтобы получился палиндром.

Примеры:

"aaab" → true # можно удалить последний символ

"xyz" → false # нельзя превратить в палиндром удалением одного символа

Что требуется

- Требуется решение за линейное время с константой дополнительной памяти. Хорошим бонусом будет, если кандидат обойдется без лишней копипасты при проверке двух случаев.

- Решение:

1. Если строка уже является палиндромом, то ответ всегда положительный (достаточно удалить центральный или один из центральных символов).

2. В противном случае, если начать проверять строку на палиндромность с левого и правого концов, можно найти первое несовпадение. В случае положительного ответа один из символов в

несовпадении должен был быть удалён, после чего оставшаяся строка должна быть палиндромом. Таким образом задача сводится к поиску такого несовпадения и проверке двух случаев.

Где ошибаются

- Лишняя копия при проверке двух случаев (пропустить левый / пропустить правый символ).

Похожие задачи для тренировки

- LeetCode 680 *Valid Palindrome II* (ровно она) — <https://leetcode.com/problems/valid-palindrome-ii/>
- CodeRun 304 *Проверка палиндрома* (базовая версия) — <https://coderun.yandex.ru/problem/palindroming-check>

#Два указателя

Два указателя

Техника двух указателей: сходящиеся, догоняющие и слияние по указателям.

3 задачи · описание, разбор подхода и типичные ошибки

1 Найти K ближайших к заданному элементов в отсортированном массиве

#два-указателя

#сортированный-массив

#слияние

Дан отсортированный по неубыванию массив целых чисел `a`, индекс элемента `index` и целое число `k`.

Необходимо вернуть в любом порядке `k` чисел из массива, которые являются ближайшими по значению к элементу `a[index]`.

Ограничения:

- Размер массива $1 \leq N \leq 10^6$;
- Элементы массива: $-10^9 \leq a[i] \leq 10^9$;
- Число $0 \leq k \leq N$;
- Индекс элемента $0 \leq \text{index} < N$.

```
std::vector<int> find_k_closest(const std::vector<int>& a, size_t index, size_t k)
```

```
find_k_closest(a={2, 3, 5, 7, 11}, index=3, k=2) -> {5, 7}
```

```
find_k_closest(a={4, 12, 15, 15, 24}, index=1, k=3) -> {12, 15, 15}
```

```
find_k_closest(a={2, 3, 5, 7, 11}, index=2, k=2) -> {3, 5} или {5, 7}
```

Что требуется

- Ожидаем решение за $O(K)$: проход по массиву двумя указателями + сбор результирующего массива, по сути это sorted merge двух массивов.
- Если хочется большей однозначности, можно добавить условие, что при прочих равных предпочтение отдаётся меньшим по значению элементам.

Где ошибаются

- Выход за пределы массива с левой или с правой стороны.
- Выход за пределы массива на массиве размера 1.
- Выбирается больше элементов, чем нужно (не учитываются совпадающие по значению элементы).
- Среди двух одинаково отдалённых элементов выбирается больший (если было чёткое требование, что делать в этом случае).

Похожие задачи для тренировки

- LeetCode 658 *Find K Closest Elements* — <https://leetcode.com/problems/find-k-closest-elements/>

2 Пофильтровать отсортированный список по другому отсортированному списку

#два-указателя

#отсортированные-списки

Даны две отсортированных по неубыванию последовательности целых чисел.

Необходимо вернуть все элементы из первой последовательности, которых нет во второй.

Пример:

```
FilterSortedList([1, 2, 3], [3, 4]) → [1, 2]
```

Что требуется

- Двигаем курсоры по спискам, сравниваем элементы; хотим аккуратный код без запутывания.
- Обязательно $O(n)$ по времени и $O(1)$ дополнительной памяти.
- Про повторяющиеся элементы кандидат должен подумать сам и уточнить. Примеры (кандидату сами не даём): `FilterSortedList([1, 2, 2], [2]) → [1]`; `FilterSortedList([1, 2, 3], [2, 2, 2]) → [1, 3]`.
- Подсказки: решение с hash set — просим оптимизировать по памяти (массивы отсортированы); решение с бинарным поиском — просим оптимизировать по времени (первый массив тоже отсортирован).

Где ошибаются

- Решение больше чем за линию или с доп-памятью.
- Прекращают работу, когда закончится 2-й список, не выводя оставшиеся элементы первого. Пример: `FilterSortedList([1, 2, 3], [1]) → [2, 3]`.
- Плохо обрабатывают повторяющиеся элементы. Пример: `FilterSortedList([1, 1, 1], [1]) → []`.
- Зависают где-нибудь, если забудут двигать курсор.

Похожие задачи для тренировки

- Идеино: LeetCode 350 *Intersection of Two Arrays II* (обратная операция, два указателя по отсортированным) — <https://leetcode.com/problems/intersection-of-two-arrays-ii/>
- Точного аналога «разность отсортированных» нет — авторская.

3 Скалярное произведение сжатых векторов

#два-указателя

Даны 2 вектора целых чисел одинаковой длины, заданные в сжатой форме списками пар вида

(value, count). Например, вектор [4, 4, 5] задается как [(4, 2), (5, 1)].

Необходимо посчитать скалярное произведение заданных векторов.

Пример:

```
DotProduct([(1, 3)], [(1, 2), (10, 1)]) → 12
```

Что требуется

- Решение с двумя указателями. $O(n)$ по времени (от суммы длин данных списков, а не размера векторов), $O(1)$ по памяти.
- Идея тривиальная, основная сложность в написании.

Где ошибаются

- Нельзя изменять исходные массивы (по сути это $O(n)$ доп. памяти).
- Считаем, что результат и промежуточные вычисления влезают в `int64_t`, но будет плюсом, если кандидат про это явно спросит.

Похожие задачи для тренировки

- LeetCode 1570 *Dot Product of Two Sparse Vectors* (разреженные, идейно близко) — <https://leetcode.com/problems/dot-product-of-two-sparse-vectors/>

#Массивы

Массивы

Массивы, префиксные суммы, скользящее окно и одно-проходные алгоритмы.

6 задач · описание, разбор подхода и типичные ошибки

1 Подотрезок с суммой X

#префиксные-суммы

#хэш-таблица

Дан массив целых чисел, нужно найти непустой подотрезок (непрерывную подпоследовательность) с заданной суммой X , либо сказать, что это невозможно.

Для найденного отрезка (если он существует) следует выдать на выход индексы его концов.

Что требуется

- Можно найти любую подпоследовательность, но хотелось бы, чтобы кандидат сам про это спросил.
- Частичные суммы, далее два варианта: 1) Сортировка / `std::map`, решение за $O(n \log n)$. 2) Hash map, решение за $O(n)$.
- Если кандидат не может сам прийти к решению — подсказать, что сумму на отрезке можно считать через частичные суммы (постепенно): сначала $O(N^2)$; потом придумать, как искать сумму на отрезке быстрее, чем за линейку, какие предподсчёты для этого можно сделать.

Где ошибаются

- В массиве могут быть отрицательные числа.
- Подотрезок должен быть непустой.
- Забывают, что левая граница отрезка должна находиться слева от правой.
- Забывают, что подотрезок может включать самый первый элемент (частичная сумма равна X).
- Хорошо, если кандидат проговорит про переполнение (префиксная сумма может не поместиться в `int`) — но это не blocker.

Похожие задачи для тренировки

- LeetCode 560 *Subarray Sum Equals K* (та же идея с префиксами) — <https://leetcode.com/problems/subarray-sum-equals-k/>
- LeetCode 325 *Maximum Size Subarray Sum Equals k* — <https://leetcode.com/problems/maximum-size-subarray-sum-equals-k/>

2 Найти максимальный по длине строго монотонный подотрезок массива

#массивы

#один-проход

Дан массив чисел `a_1, a_2, ..., a_n`.

Необходимо найти монотонный подотрезок (то есть строго убывающий или строго возрастающий) максимальной длины и вернуть пару индексов его начала и конца.

Примеры:

```
[2, 7, 5, 4, 4, 3] -> {1, 3}
[1, 1] -> (1, 1) // or (0, 0)
```

Что требуется

- Решение за линейное время и константу дополнительной памяти (менять входной массив нельзя).
- Существует решение за линию за один проход, в котором необходимо поддерживать текущую монотонную последовательность и аккуратно её обновлять при переходе к следующему числу.
- Допускается также решать за два прохода, отдельно рассматривая случай возрастания и убывания.

Где ошибаются

- Большинство кандидатов почему-то предпочитают делать за один проход (и ошибаются в нём), особенно на плоских участках `a[i]==a[i+1]`.

Похожие задачи для тренировки

- LeetCode 674 *Longest Continuous Increasing Subsequence* (только возрастание) — <https://leetcode.com/problems/longest-continuous-increasing-subsequence/>
- LeetCode 978 *Longest Turbulent Subarray* (родственная) — <https://leetcode.com/problems/longest-turbulent-subarray/>

3 Максимальный подмассив единиц

#скользящее-окно

#массивы

Дан непустой массив из нулей и единиц. Нужно определить, какой максимальный по длине подынтервал единиц можно получить, удалив (пропустив) ровно один элемент массива. Вернуть 0, если такого подынтервала не существует.

Удалять один элемент из массива обязательно.

Требуется решение без дополнительной памяти — $O(1)$, и за линейное время — $O(N)$, исходный массив менять нельзя.

Дополнительные тест-кейсы: не стоит давать их сразу, но можно использовать, чтобы подсветить проблемы в коде кандидата:

```
assert(maxones({0, 0}) == 0);
assert(maxones({1, 0}) == 1);
assert(maxones({0, 1}) == 1);
assert(maxones({1, 1, 1, 0, 1, 1, 0, 0, 1, 1, 1, 1}) == 5);
assert(maxones({1, 1, 1, 0, 1, 1, 0, 1, 1, 1, 1}) == 6);
assert(maxones({1, 1, 1, 1, 0}) == 4);
assert(maxones({1, 1, 1, 1, 1}) == 4);
```

Что требуется

- В один проход, накапливаем длины последних двух последовательностей, на основании этого обновляем максимум.
- Либо: ищем паттерны 1..1-0-1..1 и 1..1, из них выбираем тот, который даст максимальную длину.
- Либо вариант «с окном»: двигаем правую часть окна (обновляя максимум), пока кол-во нулей внутри не выходит за предел (в нашем случае 1), потом двигаем левую, пока кол-во нулей не станет меньше порога. Решение обобщается на случай N возможных нулей.

Где ошибаются

- Краевой случай: последовательность единиц длиннее, чем две другие последовательности, разделённые нулём: `{1, 1, 0, 1, 0, 0, 1, 1, 1, 1, 1}`.
- Массив из всех единиц. Не удалять элемент в этом случае нельзя.

Похожие задачи для тренировки

- LeetCode 1493 *Longest Subarray of 1's After Deleting One Element* (ровно она) — <https://leetcode.com/problems/longest-subarray-of-1s-after-deleting-one-element/>

- LeetCode 1004 *Max Consecutive Ones III* (обобщение до K нулей) — <https://leetcode.com/problems/max-consecutive-ones-iii/>

4 Найти сумму единиц

#массивы

#один-проход

Дан массив целых чисел, необходимо написать метод, который найдет максимальное количество идущих подряд 1 в массиве.

Что требуется

- Один проход, O(1) памяти.

Где ошибаются

- Обработка null/пустого массива; сброс счётчика на нуле.

Пример решения

```
class Solution {
    public int findMaxConsecutiveOnes(int[] nums) {
        if (nums == null) {
            throw new IllegalArgumentException("Input array is null");
        }

        int curCount = 0;
        int maxCount = 0;

        for (int n : nums) {
            if (n == 1) {
                curCount++;
                maxCount = Math.max(maxCount, curCount);
            } else {
                curCount = 0;
            }
        }

        return maxCount;
    }
}
```

Похожие задачи для тренировки

- LeetCode 485 *Max Consecutive Ones* — <https://leetcode.com/problems/max-consecutive-ones/>

5 Разделить массив на подмассивы минимальной стоимости

#массивы

#жадность

#один-проход

Дан массив целых чисел *nums* длины *n*.

Стоимость массива значение его первого элемента. Например, стоимость массива [1,2,3] - 1, а стоимость [3, 4, 1] - 3.

Нужно разделить массив на 3 непрерывных непересекающихся подмассива.

Написать функцию, которая возвращает минимальную из возможных сумм стоимостей этих трех подмассивов.

```
min_cost([1, 2, 3, 12]) -> 6
```

т.к. можно разделить массив на следующие подмассивы: [1], [2], [3, 12]

```
min_cost([5, 4, 3]) -> 12
```

т.к. можно разделить массив на следующие подмассивы: [5], [4], [3]

```
min_cost([10, 3, 1, 1]) -> 12
```

т.к. можно разделить массив на следующие подмассивы: [10, 3], [1], [1]

Что требуется

- Решение, в котором добавляем в итоговую стоимость первый элемент, а также два минимальных элемента из массива, содержащего все элементы, кроме первого.
- Задача о поиске двух минимумов в массиве. Хочется услышать решение за $O(n)$ за один проход.

Где ошибаются

- Решение за $O(n^2)$ — не ок. Сортировка тоже не ок. Решение за $O(n)$, но с использованием двух циклов уже лучше, но хочется за один проход.

Пример решения

```
func minCost(nums []int) int {
    min1, min2 := math.MaxInt, math.MaxInt
    for _, num := range nums[1:] {
        if num < min1 {
            min1, min2 = num, min1
        } else if num < min2 {
            min2 = num
        }
    }
    return nums[0] + min1 + min2
}
```

Похожие задачи для тренировки

- LeetCode 3010 *Divide an Array Into Subarrays With Minimum Cost I* (ровно она) — <https://leetcode.com/problems/divide-an-array-into-subarrays-with-minimum-cost-i/>
- Follow-up: LeetCode 3013 *...Minimum Cost II* — <https://leetcode.com/problems/divide-an-array-into-subarrays-with-minimum-cost-ii/>

6

Удалить нули из вектора

#массивы

#два-указателя

#inplace

Дан вектор, надо удалить из него нули, сохранив порядок остальных элементов. Интересно как с использованием стандартных средств, так и без них.

Для языков типа Python или Java, где управление памятью не так очевидно, можно использовать формулировку "перенести нули в конец массива" вместо удаления.

Что требуется

- Руками за $O(n)$ времени и $O(1)$ памяти.
- Follow-up: знание стандартных языковых идиом. Например, `std::remove + vector::resize/erase` для C++.

Где ошибаются

- Решение за N^2 здесь не ОК. Если кандидат пишет через стандартные функции, надо просить написать цикл руками.

Похожие задачи для тренировки

- LeetCode 283 *Move Zeroes* (ровно она) — <https://leetcode.com/problems/move-zeroes/>
- LeetCode 27 *Remove Element* (родственная) — <https://leetcode.com/problems/remove-element/>

#Хэш-таблицы и множества

Хэш-таблицы и множества

Подсчёт, дедупликация и быстрый поиск через хэш-таблицы и множества.

4 задачи · описание, разбор подхода и типичные ошибки

1 Оптимизация маршрута 2

#хэш-таблица

#множества

#моделирование

#сетка

Дан список ненулевой длины состоящий из направлений. Направление обозначается одним из четырех символов:

L-Left

R-Right

U-Up

D-Down

Каждый элемент перемещает нас на 1 в заданном направлении.

Известно, что петли (возврат в уже посещенную точку) дают нулевое перемещение и являются пустой тратой времени. Нужно удалить из списка все петли и вернуть оптимизированный короткий маршрут, например:

`[R,D,L,U,R] -> [R]``[R,D,L,R,U,U,R] -> [R,U,R]`

Важно отметить, что цель не просто попасть в ту же самую конечную точку, но и придерживаться первоначального маршрута (не срезать по прямой):

`[D,R,U] -> [D,R,U]`

Вернуть нужно массив направлений (а не массив посещенных точек).

Ограничения: $O(N)$ по памяти, $O(N)$ по времени

Что требуется

- Хэшсет с посещёнными координатами и список координат. Как только видим, что в хэшсете что-то есть, отступаем по списку, пока не найдём ту самую координату, по пути очищая хэшсет. Просто и быстро, но почти никто не придумывает.
- Хэшмапа, в которой хранится направление, куда идти дальше из этой точки. Если вернулись в ту же точку — перезатираем новым направлением.
- Хэшмапа + вектор посещённых точек; в хэшмапе храним последний индекс точки в векторе.
- Решение не уникально: если идти с конца, можно получить другое. Самый короткий маршрут искать не требуется — годится любой.
- Полезно в начале предложить кандидату на листочке начертить какой-нибудь путь.
- Если осталось время — поговорить про поиск оптимального маршрута (shortest path).

Где ошибаются

- Складывают текущую координату в хэшмапу (и позицию в списке); как только точка есть — смотрят позицию и удаляют всё до текущей точки. При удалении можно случайно сделать квадрат, вырезать лишнее, забыть почистить хэшмапу в процессе вырезания, да и просто сложновато получается.
- Тонкость решения с хэшмапой направлений: если финальная точка маршрута находится на петле, то в ней будет записано куда идти дальше, а ведь дальше-то и не надо. Это почти все пропускают, бить за это не надо — надо просто намекнуть.

Похожие задачи для тренировки

Задачи, которые похожи на эту и можно их прорешать для тренировки:

- LeetCode 1041 *Robot Bounded In Circle* (моделирование движения) — <https://leetcode.com/problems/robot-bounded-in-circle/>
- LeetCode 1249 *Minimum Remove to Make Valid Parentheses* (стек-откат) — <https://leetcode.com/problems/minimum-remove-to-make-valid-parentheses/>

2 Удалить лишние повторения из массива

#хэш-таблица

#массивы

Дан массив, необходимо оставить только N повторений каждого числа, сохранив порядок

Что требуется

- Решение за $O(N)$ с использованием $O(N)$ памяти.
- Если кандидат справился хорошо, можно спрашивать как безопасно этот код сделать многопоточным (с использованием M потоков).

Где ошибаются

- Квадратичные решения; порча исходного порядка элементов.

Похожие задачи для тренировки

- LeetCode 26 *Remove Duplicates from Sorted Array* (частный случай $N=1$) — <https://leetcode.com/problems/remove-duplicates-from-sorted-array/>
- LeetCode 80 *Remove Duplicates from Sorted Array II* ($N=2$) — <https://leetcode.com/problems/remove-duplicates-from-sorted-array-ii/>
- «N для неотсортированного» — авторский вариант.

3 Пересечение топов двух выдач

#хэш-таблица

#множества

#массивы

Поисковая формулировка: для двух поисковых выдач, заданных массивами DocId'ов (т.е. просто векторами целых чисел) длины И, для всех K от 1 до N нужно посчитать количество общих документов в топах размера K.

Абстрактная формулировка, вариант для AA-секций:

Для двух массивов целых чисел длины N, для всех K от 1 до N, посчитать количество общих чисел на префиксах длины K.

Числа в пределах массива могут повторяться, пересечение считается без учета кратности.

```
assert(CountCommonNumbers({1, 1, 2, 3}, {2, 1, 3, 1}) == vector<int>({0, 1, 2, 3}));
```

Задачу можно давать и в других постановках:

- Числа в пределах массива могут повторяться, пересечение считается с учетом кратности (минимум из кратностей в префиксах).

```
assert(CountCommonNumbers({1, 1, 1, 1}, {1, 2, 1, 4}) == vector<int>({1, 1, 2, 2}));
```

- Все числа в каждом из массивов различные.

```
assert(CountCommonNumbers({1, 2, 3, 4}, {2, 0, 1, 3}) == vector<int>({0, 1, 2, 3}));
```

Что требуется

- Решение за линейное время с двумя хэш-сетами для левой и правой выдачи, соответственно. При решении на C вместо хэш-сета допустимо использовать lookup-таблицу в форме массива или битовой маски (64 цвета поместятся в long).

- Если повторы внутри одного вектора запрещены, то возможно решение с одним хэш-сетом всех встреченных элементов двух массивов.
- Хорошо, если кандидат сам спросит, допускаем ли мы повторы чисел внутри одного вектора.
- Можно попросить учесть повторы, чтобы усложнить задачу, либо как follow-up вопрос на AA секции для бэкендеров.

Где ошибаются

- Не уточняют про повторы внутри вектора; квадратичный подсчёт по всем префиксам.

Похожие задачи для тренировки

- LeetCode 349 *Intersection of Two Arrays* — <https://leetcode.com/problems/intersection-of-two-arrays/>
- LeetCode 350 *Intersection of Two Arrays II* (с кратностью) — <https://leetcode.com/problems/intersection-of-two-arrays-ii/>
- CodeRun 61 *Пересечение множеств* (без «по префиксам») — <https://coderun.yandex.ru/problem/intersection-sets>

4 Вертикальная ось симметрии (в оригинале без названия)

#хэш-таблица

#геометрия

#краевые-случаи

Дан массив точек с целочисленными координатами (x, y) . Определить, существует ли вертикальная прямая, делящая все точки, не лежащие на ней, на 2 симметричных относительно этой прямой набора точек.

Наборы симметричны когда каждая точка исходного массива имеет пару из другого набора.

Инварианты условия, псевдокодом:

```
isVertsym([[0, 0], [0, 1], isVertsym([[0, 0], [0, 0], isVertsym([[0, 0], [0, 0], isVertSym([]))
// true
isVertsym([[0, 0]]) // true
[1, 1], [2, 2], [3, 1], [4, 1], [4, 0]]) // true [1, 1], [2, 2], [3, 1], [4, 0], [4, 0]]) //
true [1, 1], [2, 2], [3, 1], [4, 0]]) // false
isVertsym([[0, 0], [10, 0]]) // true isVertsym([[0, 0], [11, 1]]) // false isVertsym([[0, 0],
[1, 0], [3, 0]]) // false
```

```
struct Point {
    int x, y;
};

bool HasVerticalSymLine(const std::vector<Point>& points) {
    //...
}
```

```
class Point {
    int x, y;
}

boolean hasVerticalSymLine(Point[] points) {
    //...
}
```

Что требуется

- Практически все разумные кандидаты придумывают решение; наиболее интересен код и краевые случаи.
- Ожидаем решения за $O(n)$ через hash map:

1. Найти min/max за $O(N)$.
 2. Из min/max получить ось симметрии.
 3. Положить все точки в hash.
 4. Пройтись по хешу, проверить симметричность.
- В случае AA-секций обязательно дойти с кандидатом до решения с линейной сложностью.
 - В продолжении можно поговорить про разбиение на подзадачи с помощью хэш-функции, зависящей от абсциссы и расстояния до оси. Возможно, про мапредьюс. Возможное продолжение в сторону околоархитектурной секции: что делать, если точек очень много и они не помещаются на одну машину?

Где ошибаются

- Решения за $O(n \log n)$ (через сортировку или почему-то с использованием tree map) принимать не стоит.
- Точность оси (лучше держать удвоенную координату `min_x + max_x`), дубликаты точек, точки на самой оси, переполнение — тут много подводных камней, за которые можно штрафовать.
- Критерии: Плохо — не решили задачу / не дописали код / набрали больше чем 3 штрафных балла. Хорошо — не более 3 штрафных баллов. Отлично — не более 1 штрафного балла.

Похожие задачи для тренировки

- LeetCode 356 *Line Reflection* (ровно она) — <https://leetcode.com/problems/line-reflection/>

#Сортировка и интервалы

Сортировка и интервалы

Сортировка, интервалы и scanline-подход к событиям на прямой.

4 задачи · описание, разбор подхода и типичные ошибки

1 Количество пар в массиве с разницей $\geq K$

#сортировка

#два-указателя

Дан массив интов и число $K, K \geq 0$.

Нужно найти количество пар (уникальных или не уникальных, с точки зрения их значения) элементов, разница которых $\geq K$.

Примеры:

```
k_pairs({1,2,3,4}, 3) -> 1
```

```
k_pairs({1,1,3,4}, 2) -> 4
```

```
k_pairs(), 2) -> 0
```

```
k_pairs({1,1,3,4}, 20) -> 0
```

Что требуется

- Отсортировать массив, и пройти "окном" длины K по всем элементам. Добавлять в результат надо элементы за пределами окна.
- При этом кейс (a_i, a_i) для $K=0$ - корректный, программа должна это учитывать. Пример: `assert solution([1, 1, 2, 4], 0) == 10`

t.me/postypashki_old Где ошибаются

t.me/postypashki_old

t.me/postypashki_old

- Неправильно обрабатывают $K=0$; двойной подсчёт пар.

Похожие задачи для тренировки

- LeetCode 1099 *Two Sum Less Than K* (окно/два указателя) — <https://leetcode.com/problems/two-sum-less-than-k/>
- LeetCode 719 *Find K-th Smallest Pair Distance* (на разности пар) — <https://leetcode.com/problems/find-k-th-smallest-pair-distance/>

2 Свертка списка в диапазоны

#сортировка

#строки

Дан список неотрицательных целых чисел, повторяющихся элементов в списке нет. Нужно преобразовать его в строку, сворачивая соседние по числовому ряду числа в диапазоны.

Примеры:

```
[1,4,5,2,3,9,8,11,0] => "0-5,8-9,11"
```

```
[1,4,3,2] => "1-4"
```

```
[1,4] => "1,4"
```

```
[1,2] => "1-2"
```

Что требуется

- Сортировка плюс один проход. По времени за $n \log(n)$.

Где ошибаются

1. Выход за границу массива на пустом массиве.
2. Выход за границу массива на массиве из 1 элемента (нужно сравнивать два элемента: текущий с предыдущим или текущий со следующим).
3. Забывают закрыть последний интервал или делают это неправильно.
4. Неправильно пишут интервал из одного элемента (например "2-2").
5. Забывают ставить разделители-запятые между интервалами, или ставят лишнюю в самом конце строки-результата.
6. В некоторых языках (python, java) может получиться квадратичная сложность, если складывать строки напрямую.

Похожие задачи для тренировки

- LeetCode 228 *Summary Ranges* (вход отсортирован) — <https://leetcode.com/problems/summary-ranges/>
- Codewars *Range Extraction* — точное 1:1 соответствие — <https://www.codewars.com/kata/51ba717bb08c1cd60f00002f>

3 Посетители Гостиницы

#сортировка

#scanline

#интервалы

Даны даты заезда и отъезда каждого гостя. Для каждого гостя дата заезда строго раньше даты отъезда (то есть каждый гость останавливается хотя бы на одну ночь). В пределах одного дня считается, что сначала старые гости выезжают, а затем въезжают новые. Найти максимальное число постояльцев, которые одновременно проживали в гостинице (считаем, что измерение количества постояльцев происходит в конце дня).

Дополнительный вопрос: в какие дни гостиница была наиболее загружена? Здорово, если кандидат сам догадается вернуть ответ не в виде списка дней, а в виде списка интервалов.

Что требуется

1. Решение через сортировку начал и концов отрезков. Общая сложность $O(N \log N)$.
2. Оценить сложность.
3. Рассуждения про краевые условия и особые точки (например, что интервалы могут иметь только одну общую точку).

Где ошибаются

- Порядок событий в один день (сначала выезды -1 , потом въезды $+1$).

Похожие задачи для тренировки

- LeetCode 253 *Meeting Rooms II* (макс. одновременных) — <https://leetcode.com/problems/meeting-rooms-ii/>
- LeetCode 1854 *Maximum Population Year* (тот же sweep) — <https://leetcode.com/problems/maximum-population-year/>

4 Найти пересекающиеся встречи

#интервалы

#scanline

```
public class Meeting {  
    long from;
```

```
    long to;
}

public List<Meeting> crossing (List<Meeting> meetings) ...
```

Что требуется

- Базовое решение — перебор пар (см. исходник ниже). Хорошо предложить оптимизацию через сортировку концов интервалов до $O(N \log N)$.

Где ошибаются

- equals/hashCode для Meeting (в исходнике помечено TODO); дубликаты в результате.

Пример решения

```
public class Meeting {
    long from;
    long to;
    boolean isCrossing(Meeting other) {
        return !(other.from > this.to || other.to < this.from);
    }
    // Accessors
    // TODO: equals+hashCode
}

public List<Meeting> crossing(List<Meeting> meetings) {
    Set<Meeting> res = new HashSet<>();

    for (Meeting m1 : meetings) {
        for (Meeting m2 : meetings) {
            if (m1 != m2) {
                if (m1.isCrossing(m2)) {
                    res.add(m1);
                    res.add(m2);
                }
            }
        }
    }
    return new ArrayList(res);
}
```

Похожие задачи для тренировки

- LeetCode 252 *Meeting Rooms* (есть ли пересечение) — <https://leetcode.com/problems/meeting-rooms/>
- LeetCode 56 *Merge Intervals* (класс задач про интервалы) — <https://leetcode.com/problems/merge-intervals/>

#Поиск и слияние

Поиск и слияние

Бинарный поиск и слияние отсортированных структур.

2 задачи · описание, разбор подхода и типичные ошибки

1 К ближайших к X (в оригинале без названия)

#бинарный-поиск

#сортированный-массив

Дан отсортированный массив целых чисел, целое число K и X.

Необходимо вернуть к чисел из массива в порядке возрастания, которые являются ближайшими к X. Примечание: при прочих равных предпочтение отдается наименьшим элементам массива.

Предполагается:

- эл-ты массива и X не больше 10^4
- K не больше размера массива
- размер массива не более 10^6

Пример 1.

Input: [1,2,3,4,5], k=4, x=3 Output: [1,2,3,4]

Пример 2.

Input: [1,2,3,4,5], k=4, x=-1

Output: [1,2,3,4]

Что требуется

- `lower_bound` + разбор окна кандидатов вокруг найденной позиции, затем сортировка результата.

Где ошибаются

- Границы окна (выход за пределы), выбор большего из равноудалённых при требовании «предпочитать меньшие».

Пример решения

```
class Solution {
public:
    vector<int> findClosestElements(vector<int>& arr, int k, int x) {
        auto it = lower_bound(arr.begin(), arr.end(), x);
        size_t pos = std::distance(arr.begin(), it);

        size_t start_pos = pos >= k ? pos - k : 0;
        size_t end_pos = pos + k + 1 <= arr.size() ? pos + k + 1 : arr.size();
        vector<pair<int, int>> data(end_pos - start_pos);

        for (size_t i = start_pos; i < end_pos; ++i) {
            data[i - start_pos] = {std::abs(arr[i] - x), arr[i]};
        }
        std::sort(data.begin(), data.end());

        vector<int> result;
        result.reserve(k);
        std::transform(
            data.begin(), data.begin() + k, back_inserter(result),
            [](const auto& item) { return item.second; }
        );
        std::sort(result.begin(), result.end());
        return result;
    }
};
```

```
}  
};
```

Похожие задачи для тренировки

- LeetCode 658 *Find K Closest Elements* (это ровно она) — <https://leetcode.com/problems/find-k-closest-elements/>

2 Сложение ступенчатых графиков

#слияние

#два-указателя

#куча

Есть два ступенчатых графика некоторых наблюдаемых величин, заданных отсортированными списками координат начала ступенек (`<время измерения>`, `<значение>`). Подразумевается, что величина сохраняет своё значение между измерениями. До первого измерения величина равна нулю.

Требуется построить новый график, который будет являться суммой двух представленных. Т.е. построить график, на котором для каждой точки (измерения) из первого и второго графика есть соответствующая точка.

Напишите функцию, которая по двум заданным массивам измерений вернёт сумму двух представленных массивов.

```
struct Point {  
    int time;  
    int value;  
};  
  
std::vector<Point> sum_series(const std::vector<Point>& a, const std::vector<Point>& b)  
  
sum_series(a={{1, 2}, {5, 1}}, b={{2, 4}, {3, 6}, {9, 7}}) -> {{1, 2}, {2, 6}, {3, 8},  
    {5, 7},
```

пример дописать до конца нужно.

Возможная мотивация для данной задачи такая: есть два ряда чисел замеры потребления CPU двумя разными процессами (возможно, на разных хостах). Хотим получить суммарный ряд потребления CPU обоими процессами.

Что требуется

- Это задача про слияние списков. Нужно в процессе слияния поддерживать текущую сумму наблюдаемых значений и объединять повторяющиеся точки.
- Для двух графиков (списков) решение должно быть за $O(N)$.
- Усложнения для не-АА-секций: не требовать различных таймстемпов и требовать, чтобы в результирующем графике соседние значения были различными; написать сложение произвольного числа графиков (за $O(N \cdot \log(K))$ кучей текущих элементов; при малом K — просто за линию выбирать минимум на каждом шаге, $O(N \cdot K)$).

Где ошибаются

1. Некорректная обработка ситуации, когда один из списков уже закончился.
2. Многие пытаются хранить текущее значение результирующего графика в отдельной переменной и обновлять его — это ведёт к серьёзному усложнению кода, лучше попросить максимально простое решение.
3. Некоторые пытаются перевести график в набор отрезков с постоянным значением и после этого работать с отрезками — данный подход ещё более сложен, стоит отсекать.

Похожие задачи для тренировки

- LeetCode 21 *Merge Two Sorted Lists* — <https://leetcode.com/problems/merge-two-sorted-lists/>
- LeetCode 23 *Merge k Sorted Lists* (follow-уп на К графиков) — <https://leetcode.com/problems/merge-k-sorted-lists/>

#Деревья

Деревья

Обходы деревьев, хэширование поддеревьев и поиск LCA.

2 задачи · описание, разбор подхода и типичные ошибки

1 Найти два одинаковых поддерева

#деревья

#хэш-таблица

#битовая-маска

#dfs

Дано бинарное дерево с выделенным корнем, в каждой вершине которого записано по одной букве A-Z.

Две вершины считаются эквивалентными, если поддеревья этих вершин содержат одинаковое множество (т.е. без учета частот) букв.

Базовая версия: нужно найти любую пару эквивалентных вершин.

Усложненная версия: нужно найти две эквивалентные вершины с максимальным суммарным размером поддеревьев (количеством дочерних узлов).

На собеседовании начинают с базовой версии и если остается время, то дают сложную версию после.

Что требуется

- Решение за $O(NK)$, где N размер дерева, K — размер алфавита.
- То есть, обход дерева с построением дескриптора (булева массива размера K) поддерева и отображения дескриптор $\rightarrow$ два максимальных поддерева с этим дескриптором. Можно поддерживать мап из дескриптора в максимальное поддерево с таким дескриптором и обновлять пару с максимальной суммой каждый раз, когда мы пытаемся обновить в мапе максимум. Так писать даже проще.
- Хороший кандидат в качестве дескриптора использует битовую маску встреченных букв и получит сложность $O(N)$.

Где ошибаются

- Используют мультимножество (с частотами) вместо множества букв; $O(N^2)$ сравнения поддеревьев.

Похожие задачи для тренировки

- LeetCode 652 *Find Duplicate Subtrees* (эквивалентность по структуре — идейно близко) — <https://leetcode.com/problems/find-duplicate-subtrees/>
- С «множеством букв» точного аналога нет — авторская.

2 Наименьший общий предок в дереве за $O(1)$ памяти

#деревья

#указатели

Дано бинарное дерево (не поиска):

```
struct Node {
    Node* parent;
    Node* left;
    Node* right;
}
```

Нужно написать функцию, которая для двух данных вершин будет возвращать их наименьшего (ближайшего) общего предка:

```
Node* lca (Node* a, Node* b);
```

Ограничение по памяти $O(1)$. Ограничение по времени кандидат должен сформулировать сам.

Что требуется

- Считаем глубину каждой вершины, потом идём двумя указателями вверх от вершин, пока они не встретятся. $O(h)$ по времени, где h - высота дерева. Обычно люди сначала предлагают этот вариант, его и стоит писать.
- Усложнённый вариант с требованием $O(d)$ по времени, где d — расстояние между вершинами: бежать вверх от каждой из двух вершин поочередно на расстояния, растущие по степеням двойки; как только придём в общего предка, задача сводится к предыдущей.

Где ошибаются

- Используют доп. память (множество предков) — нарушает $O(1)$.

Похожие задачи для тренировки

- LeetCode 1650 *Lowest Common Ancestor of a Binary Tree III* (с указателем parent, ровно она) — <https://leetcode.com/problems/lowest-common-ancestor-of-a-binary-tree-iii/>

#Проектирование и структуры данных

Проектирование и структуры данных

Проектирование структур данных и итераторов под заданный контракт.

3 задачи · описание, разбор подхода и типичные ошибки

1 Найти всплески пользовательской активности

#проектирование

#скользящее-окно

#хэш-таблица

#очередь

Есть последовательность запросов, упорядоченная по времени.
Запросы бывают двух видов:

1. Пользователь `user_id` сгенерировал событие (нажал на красную кнопку)
2. Посчитать количество пользователей, которые за последние 5 минут сгенерировали ≥ 1000 событий (нажали на красную кнопку ≥ 1000 раз).

Необходимо реализовать структуру данных, умеющую эффективно обрабатывать данные запросы.

Ограничения по времени на обработку запроса: $O(1)$ амортизированное. Требуется, чтобы константа в оценке сложности не зависела от параметров 1000 (порогового количества событий в окне) и 5 минут (ширины окна).

Можно либо просить реализовать класс:

```
class RobotStatistics {  
    void OnEvent(time_t now, int userId);  
    int GetRobot Count (time_t now);  
};
```

Либо сказать, что у нас есть последовательность событий и после обработки каждого события мы хотим печатать число роботов за последние 5 минут.

Что требуется

- Поддерживаем окно событий за 5 минут и словарь `user_id` $\rightarrow$ `count`. При очередном событии выкидываем из окна все старые события и добавляем новое, попутно обновляем словарь. Держим счётчик, который учитывает число пользователей с большими запросами.

Где ошибаются

- Не учитываем, что окно событий за последние 5 минут может в какой-то момент опустеть.
- Не очищаем из словаря пользователей, когда их счётчик запросов достигает 0 (на очень длинном потоке будет непрерывно отжираться память). Это, наверное, нестрашная проблема для стажёра, но с сильным кандидатом стоит обсудить.
- Аналогично предыдущему: не очищаем пользователей из словаря, когда приходят только запросы на учёт событий и не приходят запросы на подсчёт пользователей.

Похожие задачи для тренировки

- LeetCode 933 *Number of Recent Calls* (скользящее окно по времени) — <https://leetcode.com/problems/number-of-recent-calls/>
- LeetCode 362 *Design Hit Counter* — <https://leetcode.com/problems/design-hit-counter/>

2 Фильтрующий итератор

#проектирование

#итераторы

Напишите реализацию итератора, возвращающий только четные (по значению) числа.

Требования:

- реализовать `Iterator`
- придумать интерфейс для конечного пользователя (что принимать на вход)
- без удаления элементов

Примечание:

- допустимо рассказать, что такое итератор, и как работает `next/hasNext`, если кандидат этого не знает (без штрафа по баллам)

- для не-джавистов написать `next/hasNext`, не опираясь на итераторы из языка (в них может быть `current`, а с ним слишком легко)

Шаблон кода

Есть набор целых чисел.

Реализовать итератор, возвращающий только чётные (по значению) числа из него.

```
public class EvenIterator implements Iterator<Integer> {  
    public boolean hasNext() {  
        return false;  
    }  
  
    public Integer next() {  
        return null;  
    }  
}
```

Что требуется

- Задача оценивается в три балла. Кандидат должен сам решить, что передавать на вход; выбор должен быть адекватен.
- На вход может быть `Collection`, `Iterator`, `Iterable`.
- Реализация через итератор, решение за $O(n)$ по вычислениям, не требующее дополнительной памяти, без дублирования кода.
- Оценка: -0.5 балла за решение $O(n)$ без доп. памяти; -0.5 за отсутствие дублирования; 1 балл за корректную реализацию.
- Если прошло меньше 20 минут: просим сделать фильтрацию произвольного типа данных произвольным выражением (ожидается `Iterator + Predicate` на вход); можно спросить про решение через стримы.

Где ошибаются

- Не принимаем решение через массив на вход — слишком специфичная структура данных.
- Не принимаем решение через `List` + индекс — на `LinkedList` обход итератора будет за $O(n^2)$.
- Не принимаем решение, требующее $O(n)$ дополнительной памяти (например, предфильтрация).

Похожие задачи для тренировки

- LeetCode 281 *Zigzag Iterator* — <https://leetcode.com/problems/zigzag-iterator/>
- LeetCode 251 *Flatten 2D Vector* — <https://leetcode.com/problems/flatten-2d-vector/>
- LeetCode 341 *Flatten Nested List Iterator* — <https://leetcode.com/problems/flatten-nested-list-iterator/>

3 Написать стек и очередь на массивах + тесты

#структуры-данных

#стек

#очередь

1. На бумаге написать реализацию стека и очереди конечного размера с использованием массивов (можно только стек или только очередь).
2. Написать тесты на полученный код

Что требуется

- Корректные push/pop/peek/isFull/isEmpty, обработка переполнения/пустоты.
- Осмысленные тесты: пустой pop/peek → исключение; push до заполнения; push сверх лимита → исключение; LIFO-порядок; чередование push/pop.

Где ошибаются

- Ошибки в граничных условиях (isEmpty , isFull), необработанные переполнение/пустота.

Пример решения

```
public class Stack {
    private int maxSize;
    private int[] dataArray;
    private int top = -1;

    public Stack(int maxSize) {
        this.maxSize = maxSize;
        dataArray = new int[maxSize];
    }

    public void push(int data) {
        if (!isFull()) {
            top++;
            dataArray[top] = data;
        } else {
            throw new IndexOutOfBoundsException("The stack is full");
        }
    }

    public int pop() {
        if (!isEmpty()) {
            return dataArray[top--];
        } else {
            throw new IndexOutOfBoundsException("The stack is empty");
        }
    }

    public int peek() {
        return dataArray[top];
    }

    public boolean isFull() {
        return top == maxSize - 1;
    }

    public boolean isEmpty() {
        return top == -1;
    }
}
```

Похожие задачи для тренировки

- LeetCode 232 *Implement Queue using Stacks* — <https://leetcode.com/problems/implement-queue-using-stacks/>
- LeetCode 225 *Implement Stack using Queues* — <https://leetcode.com/problems/implement-stack-using-queues/>
- LeetCode 622 *Design Circular Queue* — <https://leetcode.com/problems/design-circular-queue/>

ГОТОВЬСЯ С ЭКСПЕРТАМИ

Хочешь уверенно проходить алгосекции?

Мы в ПОСТУПАШКАХ готовим к алгоритмическим собеседованиям в топовые компании — от базы до самых жёстких секций. Выбирай курс под свой уровень.

Алгоритмы СТАРТ

Если ты только вкатываешься в алгоритмы — начни с базы: структуры данных, сложность, основные приёмы и первые задачи. Соберёшь фундамент, чтобы не плыть на собеседовании.

Смотреть курс СТАРТ →

Алгоритмы ХАРД

Если база уже есть и хочется брать сложные секции — разбираем продвинутые техники, хитрые задачи и подготовку к хард-собеседованиям на сильные грейды.

Смотреть курс ХАРД →

Остались вопросы?

Поможем выбрать направление и ответим на всё — просто напишите нашему менеджеру. И подписывайтесь на канал, чтобы не пропускать разборы и задачи.

Написать менеджеру · @menshe_treh

Подписаться на канал · @postypashki_old